

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Курсовая работа

по дисциплине: «Нейронные сети и глубокое обучение»

на тему: «Сравнительный анализ стандартных и depthwise separable свёрток
в архитектуре ResNet-18 на примере классификации CIFAR-10»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

Профессор, д.т.н. _____ Седых И.А.

Липецк, 2026

Задание кафедры

Студенту: Станиславчуку С.М. группы М-РИТ-25-1.

Тема: «Сравнительный анализ стандартных и depthwise separable свёрток в архитектуре ResNet-18 на примере классификации CIFAR-10».

Содержание расчетно-пояснительной записки:

1. Титульный лист
2. Задание кафедры
3. Аннотация
4. Оглавление
5. Введение
6. Основная часть
7. Заключение
8. Список источников
9. Приложение

Дата выдачи задания: _____

Срок сдачи работы: _____

Руководитель: _____ Седых И.А.

Задание принял к исполнению: _____ Станиславчук С.М.

Аннотация

Тема курсового проекта: «Сравнительный анализ стандартных и depthwise separable свёрток в архитектуре ResNet-18 на примере классификации CIFAR-10».

С. numpages; Ил. numPics; Табл. Numtable; Литература назв. numLit; Приложение numПриложений;

В данной курсовой работе исследуется эффективность замены стандартных свёрточных слоёв на разделенные по глубине (depthwise separable convolutions) в архитектуре ResNet-18. Обучение и оценка проводятся на датасете CIFAR-10. Реализованы два варианта модели: стандартный ResNet-18 и модифицированный, в котором все свёртки заменены на разделенные-по-глубине блоки. Проведено сравнение по четырём ключевым метрикам: точность классификации на тестовой выборке, количество обучаемых параметров, размер сериализованной модели и влияние на CPU. Результаты показывают, что depthwise separable вариант существенно сокращает число параметров, уменьшает размер модели и ускоряет инференс на CPU при небольшом падении точности. Работа включает обзор литературы по эффективным архитектурам свёрточных нейронных сетей, описание реализованной архитектуры, методики обучения и бенчмаркинга, анализ полученных результатов.

Содержание

1. Введение 6
   1.1 Актуальность 6
   1.2 Цель работы 7
   1.3 Задачи 7
2. Обзор литературы 8
   2.1 Стандартные свёрточные нейронные сети 8
   2.2 Depthwise Separable Convolution 8
   2.3 MobileNet 9
   2.4 Xception 10
   2.5 ResNet и остаточное обучение 10
   2.6 EfficientNet и NAS-архитектуры 10
   2.7 Другие подходы к повышению эффективности CNN 11
   2.8 Сравнительная таблица подходов 11
3. Исходный датасет 13
   3.1 Предобработка данных 13
4. Программа для обучения модели 14
   4.1 Архитектура ResNet-18 14
   4.2 Depthwise Separable Convolution 15
   4.3 Процесс обучения 16
   4.4 Методика бенчмарка 17
   4.5 Результаты экспериментов 17
5. Вывод 18
6. Список источников 20

1. Введение

1.1. Актуальность

Свёрточные нейронные сети (CNN) являются основой современных систем компьютерного зрения, обеспечивая высокое качество в задачах классификации, детекции и сегментации изображений. Однако рост точности моделей часто сопровождается значительным увеличением их вычислительной сложности и объёма памяти, необходимого для хранения параметров. Это создаёт серьёзные ограничения при развёртывании CNN на устройствах с ограниченными ресурсами: мобильных телефонах, встраиваемых системах, одноплатных компьютерах и CPU-серверах без GPU-ускорения.

Одним из наиболее успешных направлений повышения эффективности CNN является использование depthwise separable convolutions [2, 3], впервые получивших широкую известность в архитектуре MobileNet. Идея заключается в факторизации стандартной свёртки на две операции: depthwise (пространственная свёртка, применяемая независимо к каждому каналу) и pointwise (свёртка 1×1, смешивающая каналы). Такой подход позволяет радикально сократить как число параметров, так и количество вычислительных операций при незначительном снижении точности.

Архитектура ResNet [1] с остаточными связями (skip-connections) стала де-факто стандартом для многих задач компьютерного зрения благодаря способности обучать очень глубокие сети без деградации качества. Сочетание идеологии ResNet с depthwise separable свёртками представляет практический интерес: позволяет ли такая модификация сохранить преимущества остаточного обучения при существенном сокращении вычислительных затрат?

1.2. Цель работы

Целью данной курсовой работы является сравнительный анализ эффективности стандартных и depthwise separable свёрток в архитектуре ResNet-18 на примере задачи классификации изображений датасета CIFAR-10.

1.3. Задачи

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить архитектуру ResNet-18, принцип работы остаточных связей и адаптацию сети для входного разрешения 32×32 (CIFAR-10).
2. Изучить устройство depthwise separable convolution в стиле MobileNet и её теоретический выигрыш по числу параметров и операций.
3. Реализовать на PyTorch два варианта ResNet-18: стандартный и с заменой всех свёрток 3×3 и 1×1 на depthwise separable блоки.
4. Обучить обе модели на датасете CIFAR-10 в идентичных условиях (гиперпараметры, аугментация, планировщик learning rate).
5. Выполнить бенчмарк на CPU: измерить количество параметров, размер сериализованной модели и латентность инференса для каждого варианта.

6. Провести сравнительный анализ полученных метрик и сделать вывод о компромиссе между точностью и эффективностью.

2. Обзор литературы

2.1. Стандартные свёрточные нейронные сети

Традиционные свёрточные нейронные сети, начиная с архитектуры LeNet-5 [7] и AlexNet [8], используют полносвязные по каналам свёрточные слои. В стандартном свёрточном слое каждый выходной канал формируется как взвешенная сумма всех входных каналов, свёрнутых с соответствующим ядром. Для слоя с C_{in} входными каналами, C_{out} выходными каналами и ядром размера $k \times k$ число параметров составляет:

$$P_{std} = k^2 \cdot C_{in} \cdot C_{out}. \tag{1}$$

Такой подход обеспечивает богатое представление признаков за счёт полного взаимодействия между каналами, однако квадратичная зависимость от числа каналов приводит к быстрому росту параметров и вычислений при масштабировании сети.

2.2. Depthwise Separable Convolution

Depthwise separable convolution [2, 3] делит стандартную свёртку на два этапа:

1. Depthwise convolution — свёртка $k \times k$, применяемая независимо к каждому входному каналу ($\texttt{groups} = C_{in}$). Количество выходных каналов равно количеству входных. Этот этап выполняет пространственную фильтрацию без смешивания каналов.

2. Pointwise convolution — свёртка 1×1, смешивающая информацию между каналами и изменяющая их количество до требуемого C_{out}.

Суммарное число параметров depthwise separable блока составляет:

$$P_{ds} = k^2 \cdot C_{in} + C_{in} \cdot C_{out}. \tag{2}$$

Отношение числа параметров стандартной свёртки к depthwise separable:

$$\frac{P_{std}}{P_{ds}} = \frac{k^2 \cdot C_{in} \cdot C_{out}}{k^2 \cdot C_{in} + C_{in} \cdot C_{out}} = \frac{k^2 \cdot C_{out}}{k^2 + C_{out}}. \tag{3}$$

При $C_{out} \gg k^2$ сокращение стремится к k^2. Для типичного случая $k = 3$ и $C_{out} = 64$ получаем $\frac{9 \cdot 64}{9+64} \approx 7.9$ раз.

2.3. MobileNet

MobileNet [2] — семейство эффективных архитектур, построенных целиком на depthwise separable свёртках. Авторы (Howard A.G. et al., 2017) продемонстрировали, что на ImageNet MobileNet достигает точности, сопоставимой с полноценными CNN (VGG-16, GoogleNet), при радикально меньшем числе параметров и вычислительных затратах. Ключевые особенности:

- все стандартные свёртки заменены на depthwise separable;
- введены два гиперпараметра — width multiplier α и resolution multiplier ρ для тонкого контроля компромисса между точностью и скоростью;
- архитектура ориентирована на мобильные и встраиваемые приложения.

2.4. Xception

Xception [3] (Chollet F., 2017) — архитектура, предложенная как экстремальная версия Inception, в которой cross-channel correlations и spatial correlations разделены полностью. В отличие от MobileNet, где depthwise идёт перед pointwise, Xception меняет порядок: сначала pointwise (1×1), затем depthwise. Авторы показали, что на ImageNet Xception превосходит Inception V3 при меньшем числе параметров.

2.5. ResNet и остаточное обучение

ResNet [1] (He K. et al., 2016) решает проблему деградации точности при увеличении глубины сети через механизм skip-connections (остаточных связей). Вместо того чтобы обучать прямое отображение $\mathcal{H}(x)$, остаточный блок обучает невязку $\mathcal{F}(x) = \mathcal{H}(x) - x$, после чего выход формируется как $\mathcal{H}(x) = \mathcal{F}(x) + x$. Это позволяет эффективно обучать сети глубиной 50, 101 и 152 слоя.

ResNet-18 — облегчённая версия для менее ресурсоёмких задач, состоящая из четырёх групп остаточных блоков с конфигурацией каналов $\mathbf{[64, 128, 256, 512]}$ и двумя BasicBlock в каждой группе.

2.6. EfficientNet и NAS-архитектуры

EfficientNet [5] (Tan M., Le Q.V., 2019) использует нейро-архитектурный поиск (NAS) для нахождения оптимального баланса между глубиной, шириной и разрешением сети. EfficientNet-B0 активно применяет depthwise separable свёртки в сочетании с механизмом squeeze-and-excitation (SE). Результаты показывают state-of-the-art точность при значительно меньших вычислительных затратах по сравнению с предшественниками.

2.7. Другие подходы к повышению эффективности CNN

Помимо depthwise separable свёрток, существует ряд альтернативных методов:

- Group convolution — обобщение depthwise convolution, где каналы разбиваются на G групп. При $G = C_{in}$ сводится к depthwise convolution. Используется в ResNeXt [6].
- Channel pruning — удаление наименее значимых каналов после обучения. Позволяет сократить модель без изменения архитектуры.
- Quantization — переход от 32-битных весов к 8-битным или даже бинарным, что снижает объём памяти и ускоряет вычисления за счёт целочисленной арифметики.
- Knowledge Distillation — обучение компактной модели (student) на выходах большой модели (teacher), позволяющее малой сети воспроизводить поведение большой.

2.8. Сравнительная таблица подходов

В таблице 1 приведено сравнение рассмотренных подходов по ключевым характеристикам.

Таблица 1 — Сравнение подходов к повышению эффективности CNN

Подход	Сокращение параметров	Сохранение точности	Сложность реализации
Depthwise Separable (MobileNet)	$\sim 8\times$	высокая	средняя
Group Convolution (ResNeXt)	$\sim G\times$	высокая	низкая
Channel Pruning	2–5×	средняя	высокая
Quantization (INT8)	4× (память)	высокая	средняя
Knowledge Distillation	варьируется	средняя	средняя

Среди рассмотренных подходов depthwise separable convolution выделяется оптимальным балансом между сокращением параметров, сохранением точности и простотой реализации, что обуславливает выбор данного метода для модификации ResNet-18 в рамках настоящей работы.

3. Исходный датасет

Для обучения и тестирования используется датасет CIFAR-10 [4] — один из стандартных бенчмарков в задачах классификации изображений. Датасет содержит $60\,000$ цветных изображений размера 32×32 пикселя, равномерно распределённых по 10 классам: самолёт, автомобиль, птица, кот, олень, собака, лягушка, лошадь, корабль, грузовик.

- Обучающая выборка: $50\,000$ изображений (по $5\,000$ на класс).
- Тестовая выборка: $10\,000$ изображений (по $1\,000$ на класс).
- Формат: трёхканальные изображения RGB, значения пикселей в диапазоне $[0, 255]$.

3.1. Предобработка данных

1. Нормализация: значения пикселей приводятся к диапазону $[0, 1]$ делением на 255, затем нормализуются по каналам со средними $(0.4914, 0.4822, 0.4465)$ и стандартными отклонениями $(0.2470, 0.2435, 0.2616)$ — значения, вычисленные по обучающей выборке CIFAR-10.
2. Аугментация (только для обучения):
    - RandomCrop: случайная обрезка 32×32 с паддингом 4 пикселя.
    - RandomHorizontalFlip: случайное отражение по горизонтали с вероятностью 0.5.
3. Для тестовой выборки аугментация не применяется — только нормализация.

Загрузка данных осуществляется через библиотеку `torchvision.datasets.CIFAR10` с автоматической загрузкой при первом обращении. Данные подаются в модель батчами размера 128.

4. Программа для обучения модели

4.1. Архитектура ResNet-18

В качестве базовой архитектуры используется ResNet-18 [1], адаптированный для входного разрешения 32×32 (CIFAR-10). Стандартная реализация ResNet-18 для ImageNet (224×224) использует начальную свёртку 7×7 со страйдом 2 и слой MaxPool, что для CIFAR-10 привело бы к чрезмерно агрессивному уменьшению пространственного разрешения. Поэтому в адаптированной версии:

- Начальная свёртка заменена на 3×3 со страйдом 1 и паддингом 1.
- Слой MaxPool после начальной свёртки заменён на тождественное отображение (`nn.Identity`).

Архитектура состоит из следующих компонентов:

1. Начальный блок: Conv2d($3 \to 64$, 3×3, stride $= 1$, padding $= 1$) $\to$ BatchNorm2d $\to$ ReLU.
2. Четыре Residual-блока (`layer1`–`layer4`) с конфигурацией каналов $[64, 128, 256, 512]$ и числом BasicBlock-ов $[2, 2, 2, 2]$ соответственно. В блоках `layer2`–`layer4` применяется страйд 2 для понижения пространственного разрешения.
3. BasicBlock: два свёрточных слоя 3×3 с BatchNorm и ReLU между ними, а также skip-connection (остаточная связь), прибавляющая вход блока к его выходу. Если размерности входа

и выхода не совпадают (из-за изменения числа каналов или страйда), к skip-connection применяется понижающая свёртка 1×1 с BatchNorm.

4. Классификатор: AdaptiveAvgPool2d(1×1) → Flatten → Linear($512 \to 10$).

Структура BasicBlock представлена на рисунке 1.

BasicBlock
Вход x
→ Conv2d($C_{in} \to C_{out}$, 3×3) → BatchNorm → ReLU
→ Conv2d($C_{out} \to C_{out}$, 3×3) → BatchNorm
→ сложение с skip-connection(x) → ReLU → выход

Рисунок 1 — Структура BasicBlock — базового остаточного блока ResNet-18.

4.2. Depthwise Separable Convolution

В модифицированной версии модели все стандартные свёртки 3×3 и 1×1 заменены на depthwise separable convolution [2] — двухэтапную операцию:

1. Depthwise convolution: свёртка $k \times k$, применяемая независимо к каждому входному каналу (`groups` = `in_channels`). Количество выходных каналов равно количеству входных. Не смешивает информацию между каналами.

2. Pointwise convolution: свёртка 1×1, смешивающая каналы и изменяющая их количество до требуемого выходного.

Такая замена радикально сокращает число параметров и вычислительную сложность свёрточных слоёв. Для свёртки с C_{in} входными каналами, C_{out} выходными и ядром $k \times k$:

- Стандартная: $k^2 \cdot C_{in} \cdot C_{out}$ параметров.
- Depthwise separable: $k^2 \cdot C_{in} + C_{in} \cdot C_{out}$ параметров.

При $k = 3$, $C_{in} = C_{out} = 64$ сокращение составляет примерно 7.9 раз для данного слоя.

4.3. Процесс обучения

Обучение обеих моделей выполняется в идентичных условиях. Гиперпараметры:

- Оптимизатор: SGD с моментом 0.9 и weight decay $5 \cdot 10^{-4}$.
- Начальная скорость обучения (lr): 0.1.
- Планировщик lr: MultiStepLR с уменьшением в 10 раз на эпохах, соответствующих 50% и 75% общего числа эпох (для 20 эпох — milestones $[10, 15]$, $\gamma = 0.1$).
- Функция потерь: CrossEntropyLoss.
- Число эпох: 20.
- Размер батча: 128.
- Инициализация весов: Kaiming Normal (He-инициализация) для свёрток, константная (1.0 для BatchNorm-весов, 0.0 для смещений).

На каждой эпохе модель обучается на полной тренировочной выборке, после чего вычисляется точность на тестовой выборке. Сохраняется чекпоинт с наилучшей точностью. Для обеих моделей используется одно и то же начальное значение random seed (42).

4.4. Методика бенчмарка

После обучения для каждой модели выполняются замеры на CPU:

1. Подсчитывается сумма `numel()` всех тензоров с `requires_grad=True`.
2. Сериализация `state_dict()` в буфер через `torch.save`, размер буфера в мегабайтах.
3. Измеряется время одного прямого прохода (batch size = 1, вход $3 \times 32 \times 32$) на CPU. Выполняется **50** прогревочных прогонов, затем **300** замеров. Результат — среднее арифметическое $\pm$ стандартное отклонение в миллисекундах.

4.5. Результаты экспериментов

Обучение проводилось на GPU NVIDIA GTX 1660 TI (для использования CUDA ядер), бенчмарк — на CPU с фиксированным числом потоков (`OMP_NUM_THREADS=8`, `torch.set_num_threads(8)`). Результаты приведены в таблице ??.

Таблица 1 — Сравнение стандартного и depthwise separable ResNet-18

Метрика	Standard	Separable
Параметры	11 173 962	1 438 565
Размер (МБ)	42.70	5.57
Latency CPU (мс)	27.53 ± 2.33	6.93 ± 0.16
Test Accuracy (%)	91.09	89.81

Таблица 1 — Относительные показатели separable-модели

Показатель	Значение
Ускорение инференса	×3.97
Сокращение размера	×7.67
Падение точности	+1.28 п.п.

5. Вывод

Проведённое исследование демонстрирует, что замена стандартных свёрток на depthwise separable в архитектуре ResNet-18 позволяет:

1. Сократить число параметров в 7.67 раз (с 11.17 млн до 1.44 млн) и размер сериализованной модели с 42.70 МБ до 5.57 МБ.

2. Ускорить инференс на CPU в 3.97 раза (с 27.53 мс до 6.93 мс на одно изображение).

3. Сохранить точность на уровне 89.81% против 91.09% у стандартной модели — потеря составляет всего 1.28 процентных пункта.

Полученные результаты согласуются с выводами оригинальной работы по MobileNet [2]: depthwise separable convolutions обеспечивают значительный выигрыш в эффективности при незначительном снижении точности. Это делает их привлекательным выбором для развёртывания CNN на устройствах с ограниченными вычислительными ресурсами (CPU, мобильные и встраиваемые платформы).

Наблюдаемый скачок точности на 10-й эпохе (с 79.5% до 88.9% для стандартной модели) объясняется срабатыванием планировщика MultiStepLR — уменьшение learning rate в 10 раз позволяет оптимизатору совершить более точный спуск и выйти из локальных минимумов функции потерь.

6. Список источников

Список литературы

[1] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2016. — P. 770–778.

[2] Howard A.G. et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications // arXiv preprint arXiv:1704.04861. — 2017.

[3] Chollet F. Xception: Deep Learning with Depthwise Separable Convolutions // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2017. — P. 1251–1258.

[4] Krizhevsky A. Learning Multiple Layers of Features from Tiny Images // Technical Report, University of Toronto. — 2009.

[5] Tan M., Le Q.V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // Proceedings of the 36th International Conference on Machine Learning (ICML). — 2019. — P. 6105–6114.

[6] Xie S. et al. Aggregated Residual Transformations for Deep Neural Networks // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2017. — P. 1492–1500.

[7] LeCun Y. et al. Gradient-Based Learning Applied to Document Recognition // Proceedings of the IEEE. — 1998. — Vol. 86, No. 11. — P. 2278–2324.

[8] Krizhevsky A., Sutskever I., Hinton G.E. ImageNet Classification with

Deep Convolutional Neural Networks // Advances in Neural Information Processing Systems (NeurIPS). — 2012. — P. 1097–1105.

[9] CIFAR-10 — Canadian Institute for Advanced Research [Электронный ресурс] / Режим доступа к данным: https://www.cs.toronto.edu/ kriz/cifar.html, свободный.

[10] PyTorch Documentation: torchvision.datasets.CIFAR10 [Электронный ресурс] / Режим доступа к данным: https://pytorch.org/vision/stable/generated/torchvision.datasets.CIFAR10.html, свободный.

Приложение 1. Код программы

models.py — Архитектура ResNet-18 и DepthwiseSeparableConv2d

```
"""ResNet-18 for CIFAR-10: standard and depthwise-separable convolutions."""
from __future__ import annotations
import io
from typing import Type
import torch
import torch.nn as nn

class DepthwiseSeparableConv2d(nn.Module):
    """MobileNet-style block: depthwise kxk + pointwise 1x1."""
    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        kernel_size: int,
        stride: int = 1,
        padding: int = 0,
        bias: bool = False,
    ) -> None:
        super().__init__()
        self.depthwise = nn.Conv2d(
            in_channels,
            in_channels,
            kernel_size,
            stride=stride,
            padding=padding,
            groups=in_channels,
            bias=bias,
        )
        self.pointwise = nn.Conv2d(in_channels, out_channels, 1, bias=bias)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.pointwise(self.depthwise(x))

def conv3x3(
    in_channels: int,
    out_channels: int,
    stride: int = 1,
    *,
    separable: bool = False,
) -> nn.Module:
    if separable:
        return DepthwiseSeparableConv2d(
            in_channels, out_channels, 3, stride=stride, padding=1, bias=False
        )
    return nn.Conv2d(
        in_channels, out_channels, 3, stride=stride, padding=1, bias=False
    )

class BasicBlock(nn.Module):
    expansion = 1
    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        stride: int = 1,
        downsample: nn.Module | None = None,
        *,
        separable: bool = False,
    ) -> None:
        super().__init__()
        self.conv1 = conv3x3(in_channels, out_channels, stride, separable=separable)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = conv3x3(out_channels, out_channels, separable=separable)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        self.downsample = downsample
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = x
        out = self.relu(self.bn1(self.conv1(x)))
```

```
        out = self.bn2(self.conv2(out))
        if self.downsample is not None:
            identity = self.downsample(x)
        return self.relu(out + identity)

class ResNet(nn.Module):
    def __init__(
        self,
        block: Type[BasicBlock],
        layers: list[int],
        num_classes: int = 10,
        *,
        separable: bool = False,
    ) -> None:
        super().__init__()
        self.in_channels = 64
        self.separable = separable
        self.conv1 = conv3x3(3, 64, separable=separable)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.Identity()
        self.layer1 = self._make_layer(block, 64, layers[0])
        self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
        self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
        self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512 * block.expansion, num_classes)
        self._init_weights()
    def _make_layer(
        self,
        block: Type[BasicBlock],
        out_channels: int,
        blocks: int,
        stride: int = 1,
    ) -> nn.Sequential:
        downsample = None
        if stride != 1 or self.in_channels != out_channels:
            downsample = nn.Sequential(
                nn.Conv2d(self.in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels),
            )
        layers = [
            block(self.in_channels, out_channels, stride, downsample,
                  separable=self.separable)
        ]
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(
                block(self.in_channels, out_channels, separable=self.separable)
            )
        return nn.Sequential(*layers)
    def _init_weights(self) -> None:
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode="fan_out",
                                        nonlinearity="relu")
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.maxpool(x)
        x = self.layer4(self.layer3(self.layer2(self.layer1(x))))
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        return self.fc(x)

def resnet18(num_classes: int = 10, *, separable: bool = False) -> ResNet:
    return ResNet(BasicBlock, [2, 2, 2, 2], num_classes=num_classes,
                  separable=separable)
```

```
def count_parameters(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def model_size_bytes(model: nn.Module) -> int:
    buffer = io.BytesIO()
    torch.save(model.state_dict(), buffer)
    return buffer.tell()
```

train.py — Обучение и оценка

```
"""Training ResNet-18 on CIFAR-10."""
from __future__ import annotations
import time
from pathlib import Path
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from models import ResNet, resnet18

def get_dataloaders(
    data_dir: str | Path,
    batch_size: int = 128,
    num_workers: int = 2,
) -> tuple[DataLoader, DataLoader]:
    data_dir = Path(data_dir)
    normalize = transforms.Normalize(
        (0.4914, 0.4822, 0.4465),
        (0.2470, 0.2435, 0.2616),
    )
    train_tf = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ])
    test_tf = transforms.Compose([transforms.ToTensor(), normalize])
    train_set = datasets.CIFAR10(
        root=data_dir, train=True, download=True, transform=train_tf
    )
    test_set = datasets.CIFAR10(
        root=data_dir, train=False, download=True, transform=test_tf
    )
    train_loader = DataLoader(
        train_set, batch_size=batch_size, shuffle=True,
        num_workers=num_workers, pin_memory=torch.cuda.is_available(),
    )
    test_loader = DataLoader(
        test_set, batch_size=batch_size, shuffle=False,
        num_workers=num_workers, pin_memory=torch.cuda.is_available(),
    )
    return train_loader, test_loader

@torch.no_grad()
def evaluate(model: nn.Module, loader: DataLoader,
             device: torch.device) -> float:
    model.eval()
    correct = 0
    total = 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return 100.0 * correct / total

def train_model(
    model: ResNet,
    train_loader: DataLoader,
    test_loader: DataLoader,
```

```
    *,
    epochs: int = 50,
    lr: float = 0.1,
    device: torch.device | None = None,
    save_path: Path | None = None,
) -> dict[str, float]:
    device = device or torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(
        model.parameters(), lr=lr, momentum=0.9, weight_decay=5e-4
    )
    scheduler = torch.optim.lr_scheduler.MultiStepLR(
        optimizer, milestones=[int(epochs * 0.5), int(epochs * 0.75)],
        gamma=0.1
    )
    best_acc = 0.0
    t0 = time.perf_counter()
    for epoch in range(1, epochs + 1):
        model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            loss = criterion(model(images), labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        scheduler.step()
        train_loss = running_loss / len(train_loader)
        test_acc = evaluate(model, test_loader, device)
        best_acc = max(best_acc, test_acc)
        print(f"Epoch {epoch:3d}/{epochs}  loss={train_loss:.4f}  "
              f"test_acc={test_acc:.2f}%  best={best_acc:.2f}%")
    elapsed = time.perf_counter() - t0
    final_acc = evaluate(model, test_loader, device)
    if save_path is not None:
        save_path.parent.mkdir(parents=True, exist_ok=True)
        torch.save({
            "state_dict": model.state_dict(),
            "separable": model.separable,
            "test_acc": final_acc,
            "best_acc": best_acc,
        }, save_path)
    return {"final_acc": final_acc, "best_acc": best_acc,
            "train_time_s": elapsed}
```

benchmark.py — Замер латентности и размера модели

```
"""CPU latency and model size comparison."""
from __future__ import annotations
import time
from dataclasses import dataclass
import torch
import torch.nn as nn
from models import count_parameters, model_size_bytes

@dataclass
class BenchmarkResult:
    name: str
    params: int
    size_mb: float
    latency_ms: float
    latency_std_ms: float
    test_acc: float | None = None
```

```
@torch.no_grad()
def measure_latency(
    model: nn.Module,
    *,
    input_shape: tuple[int, ...] = (1, 3, 32, 32),
    warmup: int = 20,
    repeats: int = 100,
    device: torch.device | None = None,
) -> tuple[float, float]:
    device = device or torch.device("cpu")
    model = model.to(device)
    model.eval()
    x = torch.randn(*input_shape, device=device)
    for _ in range(warmup):
        model(x)
    times: list[float] = []
    for _ in range(repeats):
        t0 = time.perf_counter()
        model(x)
        times.append((time.perf_counter() - t0) * 1000.0)
    mean = sum(times) / len(times)
    variance = sum((t - mean) ** 2 for t in times) / len(times)
    return mean, variance ** 0.5

def benchmark_model(
    model: nn.Module,
    name: str,
    *,
    test_acc: float | None = None,
    device: torch.device | None = None,
    warmup: int = 20,
    repeats: int = 100,
) -> BenchmarkResult:
    device = device or torch.device("cpu")
    model_cpu = model.to(device)
    model_cpu.eval()
    lat_mean, lat_std = measure_latency(
        model_cpu, warmup=warmup, repeats=repeats, device=device,
    )
    return BenchmarkResult(
        name=name,
        params=count_parameters(model_cpu),
        size_mb=model_size_bytes(model_cpu) / (1024 * 1024),
        latency_ms=lat_mean,
        latency_std_ms=lat_std,
        test_acc=test_acc,
    )
```